

ECE264 Fall 2021 Exam 2

6:30-7:30PM, November 4

Please write your answers on the answer sheets only and **return the answer sheet**. Do not return the questions. Please keep the answer sheet clean. Do not use the answer sheet as your scratch space. The answer sheet should contain **only** the answers.

Do not write anything that is not the answers.
Otherwise, you may lose points.

Please use **DARK** ink. If your pen is too light, your answer may not be graded.

When you take this exam, you agree with the Purdue Honor Pledge:
advancing a culture of academic integrity, seeing academic dishonesty and cheating as threats to Purdue's reputation.

Value	Character	Value	Character	Value	Character
48	0	65	A	97	a
49	1	66	B	98	b
50	2	67	C	99	c
51	3	68	D	100	d
52	4	69	E	101	e
53	5	70	F	102	f
54	6	71	G	103	g
55	7	72	H	104	h
56	8	73	I	105	i
57	9	74	J	106	j
		75	K	107	k
		76	L	108	l
		77	M	109	m
		78	N	110	n
		79	O	111	o
		80	P	112	p
		81	Q	113	q
		82	R	114	r
		83	S	115	s
		84	T	116	t
		85	U	117	u
		86	V	118	v
		87	W	119	w
		88	X	120	x
		89	Y	121	y
		90	Z	122	z

1 Merge Sort

Consider the following program for merge sort.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <stdbool.h>
5
6 // input: arr is an array and its two parts
7 // arr[l..m] and arr[m+1..r] are already sorted
8 // l, m, r mean left, middle, and right respectively
9 //
10 // You can assume that l <= m <= r.
11 //
12 // output: arr is an array and all elements in arr[l..r] are sorted
13 // Do not worry about the elements in arr[0..l-1] or arr[r+1..]
14
15 void merge(int * arr, int l, int m, int r, int * mergecount)
16 {
17     (* mergecount) ++;
18     if ((l > m) || (m > r)) { return; } // one or both are empty
19     int * temparr = malloc (sizeof (* temparr) * (r - l + 1));
20     if (temparr == NULL) { return; } // malloc fail
21
22     int ind1 = l;
23     int ind2 = m + 1;
24     int ind3 = 0;
25     while ((ind1 <= m) && (ind2 <= r))
26     {
27         if (arr[ind1] < arr[ind2])
28         {
29             temparr[ind3] = arr[ind1];
30             ind1 ++;
31         }
32         else
33         {
34             temparr[ind3] = arr[ind2];
35             ind2 ++;
36         }
37         ind3 ++;
38     }
39     // copy the remaining elements
```

```

40     while (ind1 <= m)
41     {
42         temparr[ind3] = arr[ind1];
43         ind1 ++;
44         ind3 ++;
45     }
46     while (ind2 <= r)
47     {
48         temparr[ind3] = arr[ind2];
49         ind2 ++;
50         ind3 ++;
51     }
52
53     // copy temparr to array
54     ind3 = 0;
55     for (ind1 = 1; ind1 <= r; ind1 ++ )
56     {
57         arr[ind1] = temparr[ind3];
58         ind3 ++;
59     }
60     free (temparr); // release temporary memory
61 }
62
63 // merge sort has the following steps:
64 // 1. if the input array has one or no element, it is already sorted
65 // 2. break the input array into two arrays. Their sizes are the same,
66 //    or differ by one
67 // 3. send each array to the mergeSort function until the input array
68 //    is small enough (one or no element)
69 // 4. sort the two arrays
70
71 void mergeSort(int * arr, int l, int r,
72               int * sortcount, int * mergecount)
73 {
74     (* sortcount) ++;
75     if (l > r) { return; } // empty array, nothing to do
76     if (l == r) { return; } // only one element, already sorted
77     int m = (l + r)/2;
78     mergeSort(arr, l, m, sortcount, mergecount);
79     mergeSort(arr, m+1, r, sortcount, mergecount);
80     merge(arr, l, m, r, mergecount);
81 }

```

```

82
83 int main(int argc, char * * argv)
84 {
85     int size;
86     for (size = 2; size < 16; size *= 2)
87     {
88         int * arr = malloc(sizeof(int) * size);
89         int ind;
90         for (ind = 0; ind < size; ind ++)
91         {
92             arr[ind] = size - ind; // descending order
93         }
94         int sortcount = 0; // how many times mergeSort is called
95         int mergecount = 0; // how many times merge is called
96         mergeSort(arr, 0, size - 1, & sortcount, & mergecount);
97         printf("sortcount=%d, mergecount=%d\n", sortcount, mergecount);
98         free (arr);
99     }
100     return EXIT_SUCCESS;
101 }

```

The mergeSort function is called three times using arrays of sizes 2, 4, and 8. This question asks you how many times mergeSort and merge are called. This is the program's output:

```

sortcount= <<ANSWER A>>, mergecount=<<ANSWER B>>
sortcount= XXX, mergecount=XXX
sortcount= <<ANSWER C>>, mergecount= <<ANSWER D>>

```

What are the answers marked as A - D? Do not worry about XXX

This is <<ANSWER E>>: If the input array is already sorted in the ascending order, will the answer for C change? Please answer **YES** if the answer for C changes. Please answer **NO** if the answer for C does not change.

Answer: A: 3, B: 1, C: 15, D: 7, No

2 Depth First Search

Here is a portion of depth-first-search code from HW9:

```
1 bool depthFirstSolve(Maze *m, MazePos curpos, char *path, int step) {
2     // Base case 1. if curpos is at the end, append \0 to the path,
3     // return true
4     if (atEnd(curpos, m)) {
5         path[step] = '\0';
6         return true;
7     }
8     // Base case 2. if curpos is invalid (wall, out of bounds, or
9     // already visited) return false
10    if (!inBounds(curpos, m)
11        || hitWall(curpos, m)
12        || isVisited(curpos, m)) {
13        return false;
14    }
15
16    // inductive case: check each possible direction from curpos
17    char direction[4] = {NORTH, SOUTH, EAST, WEST};
18    MazePos steps[4] = {
19        {.xpos = curpos.xpos, .ypos = curpos.ypos - 1}, // NORTH
20        {.xpos = curpos.xpos, .ypos = curpos.ypos + 1}, // SOUTH
21        {.xpos = curpos.xpos + 1, .ypos = curpos.ypos}, // EAST
22        {.xpos = curpos.xpos - 1, .ypos = curpos.ypos} // WEST
23    };
24
25    for (int i = 0; i < 4; i++) {
26        // Try to make a move and see if it's successful
27        if (depthFirstSolve(m, steps[i], path, step + 1)) {
28            // If successful, record this step in the path
29            path[step] = direction[i];
30            return true;
31        }
32    }
33
34    // Set this square to visited ** THIS IS A BUG **
35    m->maze[curpos.ypos][curpos.xpos].visited = true;
36
37    // If no move is successful, return false
38    return false;
39 }
```

There is a bug in this code (line 35 should happen *before* the for loop in line 25).

ANSWER A: Consider the following maze (remember: `#` is a wall, `.` is an open space, `s` is the start space, and `e` is the end space).

```
#e##  
#.##  
#.##  
s..#
```

What will happen when you try to solve this maze? Your answer should be the number of the result below.

1. The program will say there is no solution
2. The program will go into an infinite loop/stack overflow
3. The program will say the maze is solved with the path **wsss**
4. The program will solve the maze with the path **ennn**

Answer: 4

ANSWER B: Consider the following slightly different maze (note that the start position has changed)

```
#e##  
#.##  
#.##  
#..s
```

What will happen when you try to solve this maze? Your answer should be the number of the result below.

1. The program will say there is no solution
2. The program will go into an infinite loop/stack overflow
3. The program will say the maze is solved with the path **wwnnn**
4. The program will solve the maze with the path **sssee**

Answer: 2

ANSWER C: Suppose we change lines 17–23 to:

```

17 char direction[4] = {NORTH, SOUTH, WEST, EAST};
18 MazePos steps[4] = {
19     {.xpos = curpos.xpos, .ypos = curpos.ypos - 1}, // NORTH
20     {.xpos = curpos.xpos, .ypos = curpos.ypos + 1}, // SOUTH
21     {.xpos = curpos.xpos - 1, .ypos = curpos.ypos}, // WEST
22     {.xpos = curpos.xpos + 1, .ypos = curpos.ypos} // EAST
23 };

```

In other words, the new search order is NORTH, SOUTH, WEST, EAST. What happens when we try to solve the maze from part B with the new program? Your answer should be the number of the result below.

1. The program will say there is no solution
2. The program will go into an infinite loop/stack overflow
3. The program will say the maze is solved with the path **wwnnn**
4. The program will solve the maze with the path **sssee**

Answer: 3

ANSWER D: Suppose we fix the **visited** bug (by setting the visited flag before the for loop), but then we replace lines 10–14 with the following:

```

10 if (hitWall(curpos, m)
11     || isVisited(curpos, m)) {
12
13     return false;
14 }

```

In other words, we forget to check whether the current position is within the maze boundary as part of our base case. Consider the following four mazes. Which of them could result in a segmentation fault when you run the program because the solver goes out of bounds? Your answer should be the number(s) of the maze(s) where the search goes out of bounds. (Assume the search order is as in the original code: NORTH, SOUTH, EAST, WEST)

1. **#e#**
#.#
#s#
2. **####**
...e
s###
3. **####**
e...
###s

4. .###
...e
s###

Answer: 3, 4

3 Recursive Function

Consider the following program.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int func(int n){
5     printf("Function call\n");
6     if(n <= 1){
7         return n;
8     }
9     else{
10         return func(n-1) + func(n-2);
11     }
12 }
13
14 int main(int argc, char * * argv){
15     if(argc != 2){
16         printf("Wrong number of input.\n");
17         return EXIT_FAILURE;
18     }
19     int n = (int) strtol(argv[1], NULL, 10);
20     printf("func(%d) = %d\n", n, func(n));
21     return EXIT_SUCCESS;
22 }
```

Suppose the executable program is named `recprog`.

Hint: If we run the program with

```
./recprog 1
```

the output of the program will be

```
Function call
```

```
func(1) = 1
```

ANSWER A: Run the program with

```
./recprog 8
```

What is the value of `func(8)`?

Your answer should only contain an integer, that is `func(8) = [ANSWER A]`

Answer: 21

ANSWER B: If we run the program with

```
./recprog 5
```

How many lines of **Function call** are there in the output? For example, there is one line of **Function call** in the output if we run the program with `./recprog 1`. Your answer should only contain an integer.

Answer: 15

ANSWER C: In general, denote $g(n)$ as the number of lines of **Function call** in the output by running the program with integer n . What is the relation between $g(n)$ and $g(n-1)$ and $g(n-2)$? Your answer should be the number of the statement below.

1. $g(n) = g(n-1) + g(n-2)$
2. $g(n) = g(n-1) + g(n-2) + 1$
3. $g(n) = g(n-1) + 1$
4. $g(n) = g(n-1) + 2$

Answer: 2

4 Linked List

Consider the following definition of two structures in `list.h`:

```
1 #ifndef _LIST_H
2 #define _LIST_H
3
4 typedef struct node {
5     int value;
6     struct node * next;
7 } Node;
8
9 typedef struct {
10     Node * head;
11     Node * tail;
12 } List;
13
14 #endif
```

Consider the following program.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "list.h"
4
5 // Create a new node
6 static Node * node_construct(int val){
7     Node * nd = malloc(sizeof (Node));
8     nd -> next = NULL;
9     nd -> value = val;
10    return nd;
11 }
12
13 // Insert a node to the end of a linked list
14 void list_insert(List * list, int val){
15     Node * ptr = node_construct(val);
16     if(list -> head == NULL){
17         list -> head = ptr;
18         list -> tail = ptr;
19     }
20     //----QUESTION A: Fix the following code----
21     else{
22         [ ANSWER A ] = ptr;
23         list -> tail = ptr;
24     }
```

```

25 }
26
27 // Print the values of all nodes in the linked list
28 void list_print(List * list){
29     Node * nd = list -> head;
30     while (nd != NULL){
31         printf ("%d ", nd -> value);
32         nd = nd -> next;
33     }
34     printf("\n");
35 }
36
37 // Destroy the entire linked list
38 void list_destroy(List * list){
39     Node * nd = list -> head;
40     Node * ptr;
41     while (nd != NULL){
42         ptr = nd -> next;
43         free (nd);
44         nd = ptr;
45     }
46     //----QUESTION B: Fix the following code----
47     free([ ANSWER B ]);
48 }
49
50 int main(int argc, char * * argv) {
51     if (argc < 2) {
52         printf("Not enough inputs.\n");
53         return EXIT_FAILURE;
54     }
55
56     // Create a linked list with the input values
57     List * llist = malloc(sizeof(*llist));
58     llist -> head = NULL;
59     llist -> tail = NULL;
60     int value;
61     for(int i = 1; i < argc; i++) {
62         value = (int) strtol(argv[i], NULL, 10);
63         list_insert(llist, value);
64     }
65     list_print(llist);
66     list_destroy(llist);

```

```
67     return EXIT_SUCCESS;
68 }
```

ANSWER A: Fix the code in function `list_insert`.

Answer: Acceptable answers include:

```
list -> tail -> next
(list -> tail) -> next
(*list).tail -> next
((*list).tail) -> next
```

ANSWER B: Fix the code in function `list_destroy`.

Answer: `list`

ANSWER C: Suppose the executable program is named `listprog`. If we run the program with `./listprog 10 20 30`, what is the output? Your answer should be the number of the statement below.

1. 10 20 30
2. 30 20 10
3. 10 30 20
4. 30 10 20

Answer: 1